

Todo/Task 事项管理系统深度分析

> 对 Claude Code CLI 项目中 Todo/Task 管理系统的全面剖析

> 调研日期: 2026-07-07

目录

1. 整体架构概览
2. 两个并存的系统：Todo V1 与 Task V2
3. TodoWriteTool (V1) 详解
4. Task 工具集 (V2) 详解
5. 任务生命周期管理
6. 后台任务系统 (Background Tasks)
7. 状态管理与 UI 渲染
8. 文件持久化机制
9. 动手实践：如何使用
10. 关键文件索引

1. 整体架构概览

为什么需要两套 Todo/Task 系统?

Claude Code 中存在 两套并行的任务管理系统，这是演进的结果：

- V1 (TodoWriteTool): 最初的简单方案, 数据存在内存 (AppState) 中, 仅用于当前会话
- V2 (TaskCreate/Update/List/GetTool): 后来引入的升级方案, 数据存在磁盘上, 支持跨会话持久化、团队协作、任务依赖

[illegible]

核心架构图

LLM ()

[illegible]

2. 两个并存的系统: Todo V1 与 Task V2

为什么有两个系统？

通过运行时开关 `isTodoV2Enabled()` 控制:

```
// src/utils/tasks.ts:133
export function isTodoV2Enabled(): boolean {
  if (isEnvTruthy(process.env.CLAUDE_CODE_ENABLE_TASKS)) {
    return true // ██████████ V2
  }
  return !getIsNonInteractiveSession() // ██████████ V2████ V1
}
```

- 交互式 CLI 模式 → V2 (任务工具集)
- 非交互/SDK 模式 → V1 (TodoWriteTool)
- 环境变量 CLAUDE_CODE_ENABLE_TASKS → 强制 V2

数据模型对比

维度 | V1 (TodoWriteTool) | V2 (Task 工具集)

数据位置 | `AppState.todos` (内存) | 磁盘 JSON 文件

持久化 | 否, 会话结束消失 | 是, 存储在 `~/claude/tasks/`

字段 | content, status, activeForm | id, subject, description, status, activeForm, owner, blocks, blockedBy, metadata

状态 | pending/in progress/completed | pending/in progress/completed (+ deleted 为特殊操作)

依赖管理 | 无 | blocks / blockedBy

所有者 | 无 | owner (agent 名称)

协作 | 不支持 | 支持多 agent 认领

ID 生成 | 无 (数组索引) | 自动递增数字 ID

3. TodoWriteTool (V1) 详解

怎么触发的？

LLM 模型在合适的场景下主动调用 TodoWrite 工具。触发时机由其 prompt 定义：

文件: src/tools/ToDoWriteTool/prompt.ts

触发场景 (prompt 中定义的 use cases) :

1. 复杂多步骤任务 — 3 个或以上步骤
2. 非平凡任务 — 需要规划或多个操作
3. 用户明确要求 — 用户说"用 todo list"
4. 用户列出了多个事项 — 用户说了多个任务
5. 收到新指令后 — 立即用 todo 记录需求
6. 开始工作时 — 标记为 in_progress
7. 完成任务后 — 标记完成并添加后续任务

不触发的场景：

- 单一简单任务（1 步完成）
- 纯咨询/信息性问题
- 单文件修改

怎么维护的？

```
// src/tools/ToDoWriteTool/ToDoWriteTool.ts:65
async call({ todos }, context) {
  const appState = context.getAppState()
  const todoKey = context.agentId ?? getSessionId()
  const oldTodos = appState.todos[todoKey] ?? []
  const allDone = todos.every(_ => _.status === 'completed')
  const newTodos = allDone ? [] : todos

  context.setAppState(prev => ({
    ...prev,
    todos: {
      ...prev.todos,
      [todoKey]: newTodos, // ■ agentId ■ sessionId ■■■
    },
  }))
}
```

- 每次调用时全量替换 todo 列表（不是增量更新）
- 按 agentId 或 sessionId 隔离不同的上下文
- 所有项完成时自动清空列表
- 数据存放在 AppState.todos 中

怎么添加提示词的？

```
// src/tools/ToDoWriteTool/prompt.ts:3-184
export const PROMPT = `Use this tool to create and manage a structured task list...`

export const DESCRIPTION =
  'Update the todo list for the current session...'
```

Prompt 通过场景示例 + 反例教导模型何时使用：

- 4 个正面示例（何时使用）
- 4 个反面示例（何时不使用）
- 状态管理规则
- 完成标准

验证提示（重要特性）

当关闭 3+ 个任务但没有验证步骤时，V1 会发出验证提示。该功能有 6 个门控条件：

```
// ToDoWriteTool.ts:77-86
if (
  feature('VERIFICATION_AGENT') && // ① Feature Flag ■ VERIFICATION_AGENT ■■■
  getFeatureValue_CACHED_MAY_BE_STALE('tengu_hive_evidence', false) && // ② ■■■■■■■■ tengu_hive_evidence ■■■■
  !context.agentId && // ③ ■■■■■■■■ agent ■
  allDone && // ④ ■■■■■■■■
  todos.length >= 3 && // ⑤ ■■ 3 ■■■■
```

```
!todos.some(t => /verif/i.test(t.content)) // ⑥ ████████
) {
  verificationNudgeNeeded = true
  // → █████████ VERIFICATION_AGENT █████
}
```

注意：条件 ①-③ 容易被忽略。条件 ③ 意味着子 agent 关闭 todo 列表时不会触发验证提示。

4. Task 工具集 (V2) 详解

4.1 TaskCreateTool — 创建任务

文件: src/tools/TaskCreateTool/TaskCreateTool.ts

输入参数:

```
subject: string // ██████████
description: string // ██████████
activeForm?: string // █████████ spinner████
metadata?: object // ██████████
```

执行流程:

```
██/████ TaskCreate
█
▼
████ ID (████████ ID + 1)
█
▼
██ JSON ██ → ~/.claude/tasks//.json
█
▼
██ TaskCreated hooks██████████
█
▼
hooks ██ → ████████████
hooks ██ → █████ UI
█
▼
██ { task: { id, subject } }
```

关键实现细节:

```
// src/utils/tasks.ts:284
export async function createTask(taskListId: string, taskData: Omit): Promise {
  // ████████████
  release = await lockfile.lock(lockPath, LOCK_OPTIONS)
  const highestId = await findHighestTaskId(taskListId)
  const id = String(highestId + 1)
  const task: Task = { id, ...taskData }
  await writeFile(path, jsonStringify(task, null, 2))
  notifyTasksUpdated()
  return id
}
```

4.2 TaskUpdateTool — 更新任务

文件: src/tools/TaskUpdateTool/TaskUpdateTool.ts

核心功能:

```
████████████████████████████████████████████████████████████████████████████████
█ TaskUpdate ██ █
████████████████████████████████████████████████████████████████████████████████
█ taskId: string → █████ ID █
█ subject?: string → █████ █
█ description?: → █████ █
█ activeForm?: → █████ █
█ status?: → pending/in_progress/ █
```

```
■ completed/deleted ■
■ owner?: → ■■■■■ ■
■ addBlocks?: → ■■■■■■■■■■■■ ■
■ addBlockedBy?: → ■■■■■■■■■■ ■
■ metadata?: → ■■/■■■■■■ ■
```

状态流转:

```
pending ████████→ in_progress ████████→ completed
█
██████ ██████ → deleted██████
```

智能特性:

1. 自动设置所有者 — 当 agent 将任务标记为 in_progress 时, 自动填写 owner
2. 任务完成 hooks — 标记完成时执行 TaskCompleted hooks
3. 邮箱通知 — 所有权变更时通过 mailbox 通知新 owner
4. 验证提示 — 关闭 3+ 任务时提示需要验证步骤
5. 团队成员提醒 — 完成任务时提示队友查看 TaskList

4.3 TaskListTool — 列出任务

文件: src/tools/TaskListTool/TaskListTool.ts

- 读取指定 taskListId 目录下的所有 JSON 文件
- 过滤内部任务 (metadata_internal)
- 自动过滤已完成的 blockedBy 引用
- 返回格式: #1 [pending] 任务标题 (owner) [blocked by #2]

团队工作流提示:

4.4 TaskGetTool — 获取单个任务

文件: src/tools/TaskGetTool/TaskGetTool.ts

- 按 ID 获取完整任务详情 (含 description、blocks、blockedBy, 但不含 owner/activeForm/metadata)
- 适合开始工作前获取完整上下文

4.5 工具名常量

```
// src/tools/TaskCreateTool/constants.ts
export const TASK_CREATE_TOOL_NAME = 'TaskCreate'

// ■■■■■■■■■■TaskUpdate, TaskList, TaskGet
```

5. 任务生命周期管理

V2 任务的完整生命周期

V2 任务的状态模型仅包括三种核心状态，加上一种特殊操作：

```
██████████████████████████████  
█ Pending █  
█ (██) █  
██████████████████████████████  
█
```

[illegible]

- 状态: pending → in_progress → completed
- deleted 不是独立状态, 而是 TaskUpdate 工具的特殊操作——它将任务文件从磁盘删除
- V2 中没有 failed 或 blocked 状态 (这些是后台任务系统的概念, 见第 6 章)

文件锁与并发安全

由于 V2 系统支持多 agent 并发操作，使用了文件锁机制：

```
// src/utls/tasks.ts:102-108
const LOCK_OPTIONS = {
  retries: {
    retries: 30, // ■■■■ 30 ■
    minTimeout: 5, // ■■■■ 5ms
    maxTimeout: 100, // ■■■■ 100ms
  },
}
```

锁的粒度:

- 任务列表锁 (.lock 文件): 创建任务、重置列表时使用
- 任务文件锁 (.json 文件): 更新、删除单任务时使用

High Water Mark 机制

为了防止删除任务后 ID 被重用（造成混淆），使用 high water mark 文件：

```
// src/utils/tasks.ts:92
// .highwatermark █████ ID █
// ████████████████████ max(██ID, ██ID) █████
```

```

■■■■■ ■■■■ #3 ■■■■■■
■ ■ ■
▼ ▼ ▼
■■: 1.json 2.json ■■: 1.json 2.json ■■: 1.json 2.json 4.json
■■: 0 ■■: 0 ■■: 3 ■■: 3
↑ ↑
■■■■■ ■ ID = ■■ + 1 = 4

```

6. 后台任务系统 (Background Tasks)

这是一个与 Todo/Task 不同的系统!

后台任务管理正在执行的操作（bash 命令、agent 调用），而非待办事项。

状态机

[illegible]

7 种任务类型

```
// src/Task.ts:6-13
export type TaskType =
| 'local_bash' // ■■ bash ■■
| 'local_agent' // ■■ agent ■■■
| 'remote_agent' // ■■ agent
| 'in_process_teammate' // ■■■■■■
| 'local_workflow' // ■■■■■■
| 'monitor_mcp' // MCP ■■
| 'dream' // ■■■■
```

轮询机制（事件驱动，非定时轮询）

重要纠正: pollTasks()

函数虽然定义了 (`src/utlis/task/framework.ts:255`)，但从未被任何代码调用。真正被调用的是 `generateTaskAttachments()`，它通过以下路径触发：

```

■■■■ 1: ■■■■■■
■■■■■ → processUserInput.ts:504
■→ getAttachmentMessages() → getAttachments() → ■■■■■■
getUnifiedTaskAttachments() → generateTaskAttachments()

■■■■ 2: ■■■■■■
queryLoop ■■■■ → query.ts:1580
■→ getAttachmentMessages() → getAttachments() → ■■■■■■
getUnifiedTaskAttachments() → generateTaskAttachments()

```

后台任务检查不是定时轮询，而是事件驱动的——每次工具循环迭代或用户输入时顺带检查一次。如果 AI 一次回复中调用了 5 个工具，就会检查 5 次；如果是纯聊天（无工具调用），则完全不触发。

```

■■■■ ■■→ processUserInput ■■→ ■■■■■■ (■■■■)
■
■ (AI ■■■■)
▼
queryLoop ■■■■
■→ ■■■■ #1 ■■→ ■■■■ ■■→ ■■■■■■
■→ ■■■■ #2 ■■→ ■■■■ ■■→ ■■■■■■
■→ ■■■■ #3 ■■→ ■■■■ ■■→ ■■■■■■
■→ ...

```

POLL_INTERVAL_MS = 1000 常量定义在 framework.ts:22 但没有任何代码引用它——不仅 pollTasks() 未使用它，整个代码库都没有导入这个导出常量。它属于死代码。generateTaskAttachments() 执行的流程与报告的流程图一致，只是触发方式不同。

TOCTOU 安全防护

`generateTaskAttachments` 是异步的（读取磁盘），而任务状态可能在此期间变化。使用增量补丁模式防止：

```
// framework.ts:213
export function applyTaskOffsetsAndEvictions(
  setAppState: SetAppState,
  updatedTaskOffsets: Record,
  evictedTaskIds: string[],
): void {
  setAppState(prev => {
    // ██████████
    for (const id of offsetIds) {
      const fresh = newTasks[id]
```

```
if (fresh?.status === 'running') { // ■■ still running ■■■■■■
newTasks[id] = { ...fresh, outputOffset: updatedTaskOffsets[id]! }
}
}
for (const id of evictedTaskIds) {
const fresh = newTasks[id]
if (!fresh || !isTerminalTaskStatus(fresh.status) || !fresh.notified) continue
delete newTasks[id]
}
})
}
```

输出磁盘写入

```
// src/utls/task/diskOutput.ts
export const MAX_TASK_OUTPUT_BYTES = 5 * 1024 * 1024 * 1024 // 5GB ■■
```

- 使用 O_NOFOLLOW 防止符号链接攻击 (Unix)
- 使用队列 + 单 drain 循环实现异步写入
- 超 5GB 后截断并写入 [output truncated: exceeded 5GB disk cap]
- 支持增量读取 (getTaskOutputDelta)，避免一次性加载大文件

关于 generateTaskAttachments 的补充说明

`generateTaskAttachments()` 虽然定义了 `TaskAttachment` 类型和 `attachments` 数组，但实际代码中该数组从未被填充——所有 `completed` 任务的通知由各任务类型自己的 `enqueuePendingNotification()` 回调处理，以避免重复通知。`generateTaskAttachments()` 的主要作用是计算 `updatedTaskOffsets` 和 `evictedTaskIds`，而非生成附件。

7. 状态管理与 UI 渲染

AppState 数据结构

```
// src/state/AppStateStore.ts:220
todos: { [agentId: string]: TodoList } // V1 todo ■■

// AppStateStore.ts:160
tasks: { [taskId: string]: TaskState } // ■■■■■■■■ V2 todo■
```

React 订阅模式

```
// src/state/AppState.tsx
export function useAppState(selector: (state: AppState) => T): T
export function useSetAppState(): (updater: (prev: AppState) => AppState) => void
```

- `useAppState` — 订阅特定状态片段，仅在选中值变化时重渲染
- `useSetAppState` — 获取更新函数但不订阅变化

后台任务面板

后台任务面板只显示运行中和待处理的任务（不显示已完成/失败的任务），通过 `/tasks` 命令打开：

[illegible]

注意：已完成（completed）和失败（failed/killed）的任务不会显示在此面板中——它们在达到终端状态后会被逐步驱逐出状态树。

通过 `/tasks` 命令打开：

```
// src/commands/tasks/tasks.tsx
export async function call(onDone, context) {
  return
}
```

8. 文件持久化机制

V2 任务的存储位置

```
~/claude/config/tasks/
■■■ / ← ■■■■■■
■ ■■■ .lock ← ■■■
■ ■■■ .highwatermark ← ■■ ID ■■
■ ■■■ 1.json ← ■■ #1
■ ■■■ 2.json ← ■■ #2
■ ■■■ ...
■■■ ...
```

`taskListId` 的解析优先级：

```
1. CLAUDE_CODE_TASK_LIST_ID ■■■■
2. ■■■■■■■■■■ teamName
3. getTeamName()■■■■■■■■■■
4. leaderTeamName■■■■■■■■
5. Session ID■■■■■
```

注意：虽然代码注释中提到了 `CLAUDE_CODE_TEAM_NAME`，但实际代码并未读取该环境变量。`getTeamName()` 检查的是进程内队友上下文和动态团队上下文，而非环境变量。

后台任务的输出文件

```
//tasks/
■■■ b8n3x123.output ← bash ■■■■
■■■ a9m2x456.output ← agent ■■■■
■■■ ...
```

- 使用 `O_EXCL` 防止文件已存在时被覆盖
- 使用 `O_NOFOLLOW` 防止符号链接攻击
- 使用 `session ID` 隔离不同会话

9. 动手实践：如何使用

V1 TodoWriteTool 使用方式

（模型在合适场景自动调用，用户不直接使用）

```
// LLM ■■■■■■
TodoWrite({
  todos: [
    { content: "■■■■■■■■", status: "in_progress", activeForm: "■■■■■■■■" },
    { content: "■■■■■■■■", status: "pending", activeForm: "■■■■■■■■" },
    { content: "■■■■■■■■", status: "pending", activeForm: "■■■■■■■■" },
  ]
})
```

V2 Task 工具集使用方式

```
// 1. ■■■■
TaskCreate({
  subject: "■■■■■■■■■■",
```

```

description: "■■■■■■■■■■API ■■■JWT ■■■",
activeForm: "■■■■■■■■■■"
})
// → ■■ { task: { id: "1", subject: "..." } }

// 2. ■■■■■■■■■■
TaskCreate({ subject: "■■■■■■■■", description: "...", activeForm: "..." })
// → ■■ { task: { id: "2", subject: "..." } }

TaskUpdate({ taskId: "2", addBlockedBy: ["1"] })
// → task #2 ■ task #1 ■■

// 3. ■■■■
TaskUpdate({ taskId: "1", status: "in_progress" })
// → ■■■■ owner

// 4. ■■■■■■
TaskList()
// → #1 [in_progress] ■■■■■■■■ (agent-name)
// → #2 [pending] ■■■■■■ [blocked by #1]

// 5. ■■■■■■
TaskGet({ taskId: "1" })
// → ■■■■■■

// 6. ■■■■
TaskUpdate({ taskId: "1", status: "completed" })

// 7. ■■■■
TaskUpdate({ taskId: "2", status: "deleted" })

```

10. 关键文件索引

文件 | 用途 | 重要性

`src/tools/ToDoWriteTool/ToDoWriteTool.ts` | V1 Todo 写入工具 | □□□
 `src/tools/ToDoWriteTool/prompt.ts` | V1 触发提示词 | □□□
 `src/utls/todo/types.ts` | TodoItem/ToDoList 类型定义 | □□
 `src/tools/TaskCreateTool/TaskCreateTool.ts` | V2 任务创建工具 | □□□
 `src/tools/TaskCreateTool/prompt.ts` | V2 创建触发提示词 | □□□
 `src/tools/TaskUpdateTool/TaskUpdateTool.ts` | V2 任务更新工具 | □□□
 `src/tools/TaskUpdateTool/prompt.ts` | V2 更新触发提示词 | □□□
 `src/tools/TaskListTool/TaskListTool.ts` | V2 任务列工具 | □□□
 `src/tools/TaskListTool/prompt.ts` | V2 列表触发提示词 | □□
 `src/tools/TaskGetTool/TaskGetTool.ts` | V2 任务获取工具 | □□
 `src/tools/TaskGetTool/prompt.ts` | V2 获取触发提示词 | □□
 `src/utls/tasks.ts` | 任务 CRUD 核心 + 文件锁 | □□□□□
 `src/Task.ts` | 后台任务类型定义 | □□□□
 `src/tasks.ts` | 任务类型注册表 | □□
 `src/utls/task/framework.ts` | 后台任务状态管理 (generateTaskAttachments/applyTaskOffsets, pollTasks 未使用) | □□□□□
 `src/utls/task/diskOutput.ts` | 后台任务磁盘输出 | □□□□□
 `src/state/AppStateStore.ts` | 状态定义 (todos/tasks 字段) | □□□
 `src/state/AppState.tsx` | React 状态 Hooks | □□
 `src/tools/TaskStopTool/TaskStopTool.ts` | 后台任务停止工具 | □□□
 `src/tools/TaskOutputTool/TaskOutputTool.tsx` | 后台任务输出读取 | □□
 `src/commands/tasks/tasks.tsx` | `/tasks` 命令 (面板 UI) | □□
 `src/tasks/types.ts` | TaskState 类型 | □□

总结

方面 | V1 (TodoWriteTool) | V2 (Task 工具集) | 后台任务系统

用途 | 待办清单 | 结构化任务管理 | 运行中操作监控

触发方式 | LLM 自主调用 | LLM 自主调用 | 工具循环迭代/用户输入时顺带检查

数据存储 | 内存 (AppState) | 磁盘 JSON 文件 | 内存 + 磁盘输出

持久化 | 否 | 是 | 输出持久化

并发安全 | 单线程 | 文件锁 | 增量补丁

生命周期 | 单会话 | 跨会话 | 进程级

Feature Flag | `!isTodoV2Enabled()` | `!isTodoV2Enabled()` | 始终启用